

NEHA SHARMA - U24AI050 LAB-9

26 March 2026 09:03

1. Write a C program to implement Banker's Algorithm and determine whether the system is in a safe state.

Expected Answer:

Student should write a program that:

- ☐ Takes n processes and m resources
- ☐ Inputs Allocation, Max, Available
- ☐ Computes Need
- ☐ Applies Safety Algorithm
- ☐ Prints safe sequence or unsafe state

2. Modify the Banker's Algorithm program to also process a resource request from a process.

Expected Answer:

Program should:

- ☐ Take process number and request vector
- ☐ Check:
 - o Request $\leq$ Need
 - o Request $\leq$ Available
- ☐ Temporarily allocate
- ☐ Run safety algorithm
- ☐ Grant or deny request

3. Write a program to display all matrices used in Banker's Algorithm in tabular format.

Expected Answer:

Should print:

- ☐ Allocation
- ☐ Max
- ☐ Need
- ☐ Available

4. Write a menu-driven C program for Banker's Algorithm with options:

1. Enter data
2. Display Need matrix

3. Check safe state
4. Process resource request
5. Exit

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int n, m; // n = processes, m = resources
```

```
int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX];  
int avail[MAX];
```

```
// Function to calculate NEED matrix
```

```
void calculateNeed() {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            need[i][j] = max[i][j] - alloc[i][j];  
        }  
    }  
}
```

```
// Function to display matrices
```

```
void display() {  
    printf("\nAllocation Matrix:\n");  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            printf("%d ", alloc[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
printf("\nMax Matrix:\n");  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < m; j++) {  
        printf("%d ", max[i][j]);  
    }  
    printf("\n");  
}
```

```

printf("\nNeed Matrix:\n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

printf("\nAvailable Resources:\n");
for (int i = 0; i < m; i++) {
    printf("%d ", avail[i]);
}
printf("\n");
}

```

```

// Safety Algorithm
int isSafe() {
    int work[MAX], finish[MAX] = {0};
    int safeSeq[MAX];
    int count = 0;

```

```

// Copy available to work
for (int i = 0; i < m; i++)
    work[i] = avail[i];

```

```

while (count < n) {
    int found = 0;

    for (int i = 0; i < n; i++) {
        if (!finish[i]) {
            int j;
            for (j = 0; j < m; j++) {
                if (need[i][j] > work[j])
                    break;
            }

            if (j == m) {
                for (int k = 0; k < m; k++)
                    work[k] += alloc[i][k];

```

```

        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

if (!found) {
    printf("\nSystem is NOT in safe state!\n");
    return 0;
}

printf("\nSystem is in SAFE state.\nSafe Sequence: ");
for (int i = 0; i < n; i++)
    printf("P%d ", safeSeq[i]);

printf("\n");
return 1;
}

```

// Resource Request Algorithm

```

void requestResources() {
    int process;
    int request[MAX];

    printf("Enter process number: ");
    scanf("%d", &process);

    printf("Enter request vector:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &request[i]);
    }

    // Check Request <= Need
    for (int i = 0; i < m; i++) {
        if (request[i] > need[process][i]) {
            printf("Error: Request exceeds Need!\n");
            return;
        }
    }
}

```

```

}

// Check Request <= Available
for (int i = 0; i < m; i++) {
    if (request[i] > avail[i]) {
        printf("Resources not available. Process must wait.\n");
        return;
    }
}

// Temporary allocation
for (int i = 0; i < m; i++) {
    avail[i] -= request[i];
    alloc[process][i] += request[i];
    need[process][i] -= request[i];
}

printf("Checking for safe state...\n");

if (isSafe()) {
    printf("Request GRANTED.\n");
} else {
    printf("Request DENIED. Rolling back...\n");

    // Rollback
    for (int i = 0; i < m; i++) {
        avail[i] += request[i];
        alloc[process][i] -= request[i];
        need[process][i] += request[i];
    }
}

// Main Menu
int main() {
    int choice;

    while (1) {
        printf("\n===== BANKER'S ALGORITHM MENU =====\n");
        printf("1. Enter Data\n");
    }
}

```

```
printf("2. Display Need Matrix\n");
printf("3. Check Safe State\n");
printf("4. Process Resource Request\n");
printf("5. Exit\n");
```

```
printf("Enter choice: ");
scanf("%d", &choice);
```

```
switch (choice) {
```

```
    case 1:
```

```
        printf("Enter number of processes: ");
        scanf("%d", &n);
```

```
        printf("Enter number of resources: ");
        scanf("%d", &m);
```

```
        printf("Enter Allocation Matrix:\n");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                scanf("%d", &alloc[i][j]);
```

```
        printf("Enter Max Matrix:\n");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                scanf("%d", &max[i][j]);
```

```
        printf("Enter Available Resources:\n");
        for (int i = 0; i < m; i++)
            scanf("%d", &avail[i]);
```

```
        calculateNeed();
        break;
```

```
    case 2:
```

```
        display();
        break;
```

```
    case 3:
```

```
        isSafe();
        break;
```

case 4:

```
    requestResources();  
    break;
```

case 5:

```
    return 0;
```

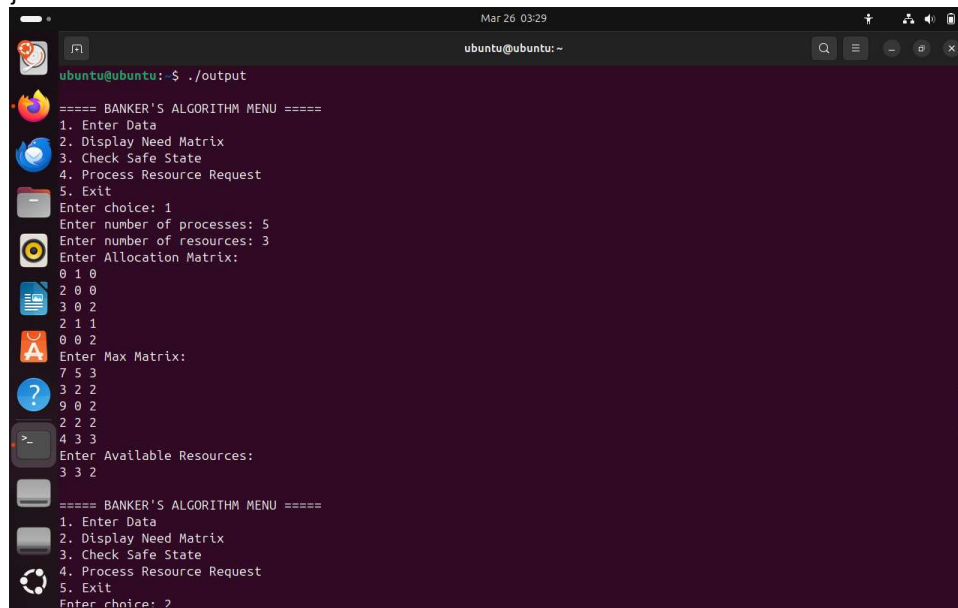
default:

```
    printf("Invalid choice!\n");
```

```
}
```

```
}
```

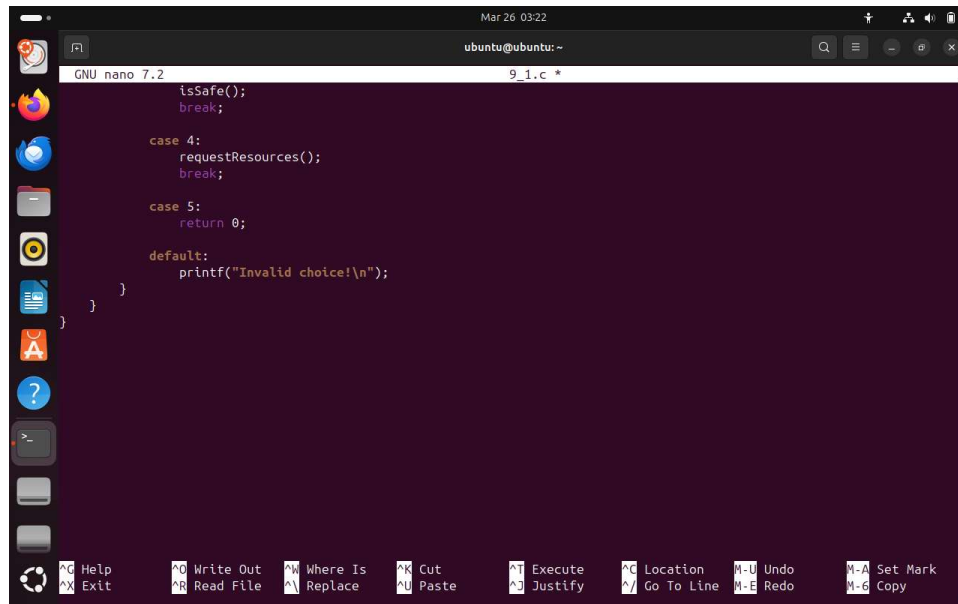
```
}
```



```
Mar 26 03:29  
ubuntu@ubuntu: ~  
ubuntu@ubuntu:~$ ./output  
===== BANKER'S ALGORITHM MENU =====  
1. Enter Data  
2. Display Need Matrix  
3. Check Safe State  
4. Process Resource Request  
5. Exit  
Enter choice: 1  
Enter number of processes: 5  
Enter number of resources: 3  
Enter Allocation Matrix:  
0 1 0  
2 0 0  
3 0 2  
2 1 1  
0 0 2  
Enter Max Matrix:  
7 5 3  
3 2 2  
9 0 2  
2 2 2  
4 3 3  
Enter Available Resources:  
3 3 2  
===== BANKER'S ALGORITHM MENU =====  
1. Enter Data  
2. Display Need Matrix  
3. Check Safe State  
4. Process Resource Request  
5. Exit  
Enter choice: 2
```

```
Mar 26 03:30
ubuntu@ubuntu:~
2. Display Need Matrix
3. Check Safe State
4. Process Resource Request
5. Exit
Enter choice: 2
Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1
Available Resources:
3 3 2
===== BANKER'S ALGORITHM MENU =====
1. Enter Data
2. Display Need Matrix
```

```
Mar 26 03:31
ubuntu@ubuntu:~
2. Display Need Matrix
3. Check Safe State
4. Process Resource Request
5. Exit
Enter choice: 3
System is in SAFE state.
Safe Sequence: P1 P3 P4 P0 P2
===== BANKER'S ALGORITHM MENU =====
1. Enter Data
2. Display Need Matrix
3. Check Safe State
4. Process Resource Request
5. Exit
Enter choice: 4
Enter process number: 1 0 2
Enter request vector:
===== BANKER'S ALGORITHM MENU =====
1. Enter Data
2. Display Need Matrix
3. Check Safe State
4. Process Resource Request
5. Exit
Enter choice: 3
System is in SAFE state.
Safe Sequence: P1 P3 P4 P0 P2
===== BANKER'S ALGORITHM MENU ===Checking for safe state...==
1. Enter Data
2. Display Need Matrix
3. Check Safe State
```

The screenshot shows a terminal window on an Ubuntu system. The window title is "Mar 26 03:22" and the user is "ubuntu@ubuntu:~". The terminal is running the GNU nano 7.2 editor, editing a file named "9_1.c". The code in the file is a C program with a switch statement. The code is as follows:

```
isSafe();
break;

case 4:
    requestResources();
    break;

case 5:
    return 0;

default:
    printf("Invalid choice!\n");
}
}
```

The terminal window has a sidebar on the left with various application icons. At the bottom, there is a menu bar with the following options:

- Help
- Exit
- Write Out
- Read File
- Where Is
- Replace
- Cut
- Paste
- Execute
- Justify
- Location
- Go To Line
- Undo
- Redo
- Set Mark
- Copy